

Four-record seals for raster restart: replay savings, commit costs, and coordinate recovery

Synthetic engineering DEMO. All observations below come from a deterministic local model and constructed fault cases. No equipment, measured workload, external dataset, or real publication is represented.

Abstract

Restarting a raster job requires a durable output boundary and a known physical position. This synthetic DEMO examines four-record block seals that preserve partial rows while retaining alternating row checkpoints. We compare them with whole-row commits and the simpler alternative of appending every record, using one 192-position geometry and twelve constructed interruption schedules. Blocks reduce payload buffering from 4096 to 1024 bytes relative to rows; record appends require 256 bytes. Blocks cost 60.0 ms more than rows without interruption. In one two-cut schedule they save 49.0 ms over rows and 40.0 ms over appends, but appends remain faster in the ten-cut case. A separate prescribed displacement control misassigns eight remaining samples when anchoring is omitted. Complete event accounting therefore exposes conditional savings and costs, while valid stored bytes do not establish position. These model results leave physical latency, storage ordering, positioning reliability and external originality untested.

1 Introduction

A power cut during a raster scan can erase two different kinds of progress: samples still in memory and the carriage's remembered position. Persisting a completed sample addresses the first loss, but does not establish where the next acquisition will occur. This distinction matters when reducing the amount of work repeated after reset. A restart procedure must first decide which logical records survived and then establish the physical coordinate associated with the first missing record. Here, acquisition and carriage motion are events in a synthetic engineering DEMO, not observations from equipment.

Whole-row commit (RC) retains sixteen 256-byte payloads before publishing a row. Its 4096-byte payload buffer motivates examining smaller persistence units, but immediate record append (AR) already offers a straightforward answer: publish each record with only 256 payload bytes buffered. The remaining question is whether four-record block sealing (BS) offers a useful intermediate choice. BS uses 1024 payload bytes and preserves completed blocks within an unfinished row, while retaining RC's alternating row checkpoints. It adds seal validation and checkpoint completion work; smaller buffering alone cannot establish its merit.

We evaluate that choice through complete event accounting rather than through the number of reacquired samples alone. All three policies traverse the same 192 positions, receive the same twelve deterministic cut schedules, and include homing and row anchoring. The comparison retains schedules in which either simple alternative wins. A separate displacement control asks whether valid stored data is sufficient to resume acquisition at the intended location. These two comparisons distinguish persistence cost from coordinate correctness without treating a checksum as a positioning guarantee.

The contribution is a source-inspectable analysis of this configured design choice: an inherited row checkpoint is extended with contiguous block recovery, and its benefit is evaluated alongside its costs and a necessary positioning action. The construction log records RC first, AR next, and BS afterward [S2]; it establishes only local implementation history. No external literature comparison, physical benchmark, or claim of historical originality is supported by the package. The resulting conclusions concern the specified DEMO and its assumptions.

2 Persistence and position recovery

The raster contains twelve rows and sixteen columns. With origin at sample center (0, 0), physical coordinates are $x = 4c$ mm and $y = 6r$ mm; the sample-center extent is 60 by 66 mm. Rows proceed in increasing r , but alternate horizontal direction. Traversal index k increases from 0 to 15 in every row: $c = k$ for even rows and $c = 15 - k$ for odd rows. Logical ordinal $q = 16r + k$ consequently increases throughout the job [S1]. Figure 1 separates this logical order from physical column order at a restart boundary.

1 Where does the next sample belong?

SYNTHETIC DEMO

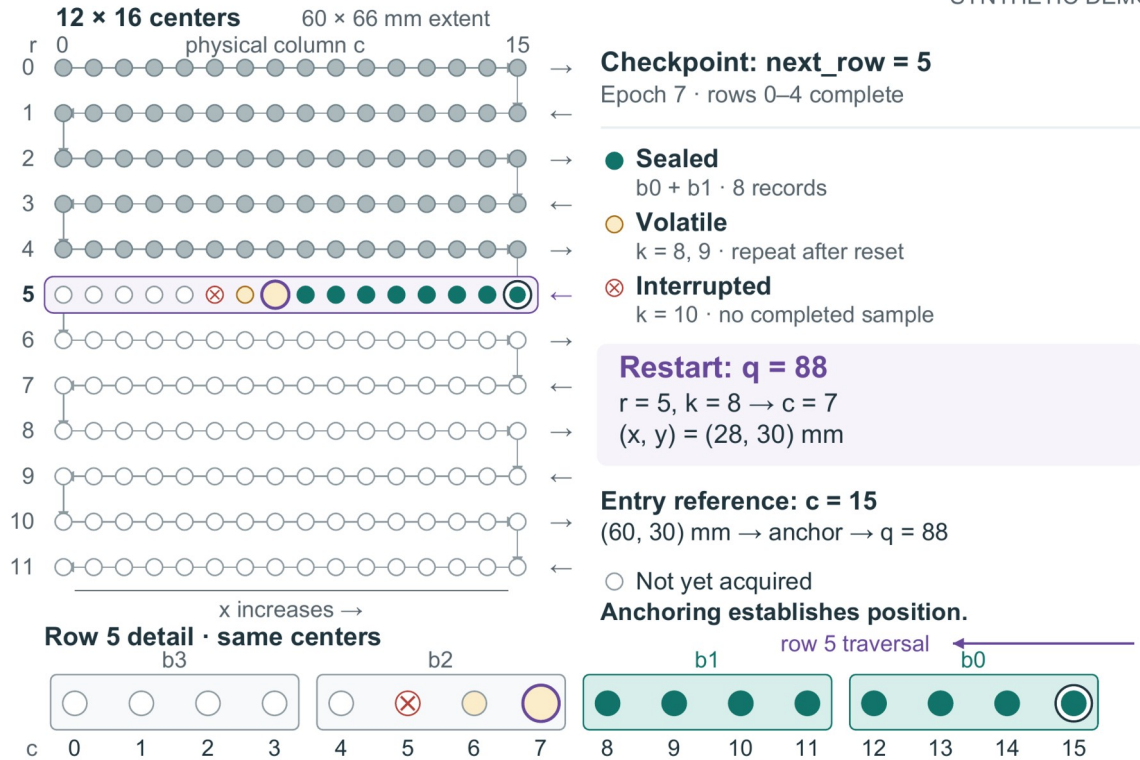


Figure 1. Constructed BS restart state, independent of the timing traces. The upper grid enumerates all 192 sample centers; the lower strip expands the same row 5, with columns increasing to the right and acquisition proceeding leftward. Rows 0–4 are complete. Sealed blocks b_0 and b_1 survive; the two volatile samples are repeated after reset. Epoch 7 and checkpoint $\text{next_row} = 5$ yield first missing $q = 88$, at (28, 30) mm. The row-entry reference remains at (60, 30) mm. Outlines and symbols supplement state colors.

Every startup, including restart, performs 120 ms of homing. Entering any row costs a further 30 ms for anchoring; after reset this includes travel from the known row-entry reference to the first missing position. The fixed cost does not depend on travel distance. Acquisition then costs 10 ms per sample, including its local move. A cut terminates the active event and discards volatile samples and the remembered coordinate. After homing, recovery lookup costs 1 ms for RC, 4 ms for AR, or 6 ms for BS; initial startup has no lookup charge.

RC stages a complete row and publishes its checkpoint in one 4 ms logical commit event. A cut before completion leaves that row unpublished. This all-or-nothing model outcome does not imply an atomic physical write of 4096 bytes. AR instead completes a 0.8 ms append after every acquisition. Each append contains 256 payload bytes and a 16-byte metadata allowance; only a completed append advances the durable contiguous record prefix. Thus AR is a strong simple comparison for BS: it preserves finer progress and requires a smaller payload buffer, at a higher configured commit cost per sample.

BS groups four successive traversal positions into a 1024-byte payload and a 24-byte seal allowance. Payload durability precedes seal publication within a combined 2 ms event. The seal identifies epoch, row, block index, record count, and payload CRC32. Four blocks complete a row; a separate 1 ms

update advances its checkpoint. BS retains RC's two alternating 64-byte slots, rather than replacing the row protocol. The slot reader chooses the valid generation with the greatest number; a torn newer slot leaves the older valid slot usable [S2, S4]. These extra persistence boundaries are the mechanism's engineering cost.

Recovery reads the valid row checkpoint, then inspects blocks $b = 0$ through 3 in that row. Acceptance requires a seal, matching epoch, row and index, four records, 1024 payload bytes, and a valid CRC. Scanning stops at the first missing or invalid block: a later valid block cannot bridge a hole. If all four blocks survive while the row checkpoint is old, recovery completes that checkpoint before entering the next row. Figure 2 shows this accepted-prefix decision separately from the operation that establishes position; both must precede subsequent acquisition.

The state in Figure 1 makes the distinction concrete. Rows 0–4 are complete and row 5 runs from column 15 toward column 0. Sealed blocks b_0 and b_1 cover $k = 0–7$, or $c = 15–8$. Acquisitions $k = 8$ and 9, at columns 7 and 6, remain volatile; $k = 10$, at column 5, is interrupted. After reset, the first missing ordinal is $q = 88$: $r = 5$, $k = 8$, $c = 7$, at $x = 28$ mm and $y = 30$ mm. Anchoring uses the row-entry reference at $c = 15$, $x = 60$ mm, before returning to that point.

2 Sealed output and position recover separately

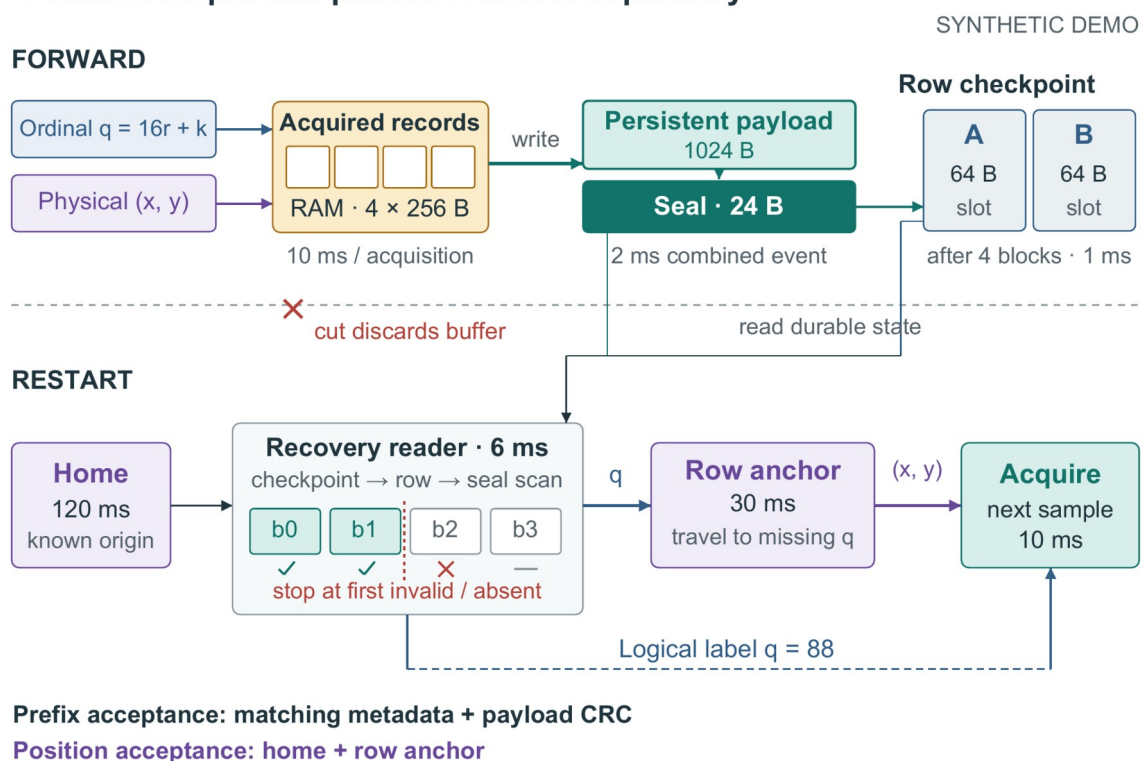


Figure 2. BS publication and restart paths. Forward operation fills four volatile records, makes the payload durable, publishes its seal, and updates an alternating row-checkpoint slot after four blocks. A cut discards the volatile buffer. Restart homes first, reads the checkpoint and validates the ordered block prefix, then anchors before acquisition. The illustrated prefix is the Figure 1 state; scanning stops at b_2 . Logical q and established physical position reach acquisition through distinct paths. If all four blocks survive, an old row checkpoint is completed before the next row is entered. Timings are assumed DEMO costs.

Equation 1 sums the configured duration d of each completed event in E_{done} and the elapsed duration τ of each interrupted event in E_{cut} . A cut exactly at an event's scheduled end wins before publication. Powered time excludes off intervals; elapsed time including them adds 40 ms per encountered cut [S1, S3]. Homing, lookup, and anchoring may themselves be interrupted, so counting only completed operations would omit consumed time. None of these three operations makes a record durable.

$$T_{power} = \sum_{e \in E_{done}} d(e) + \sum_{e \in E_{cut}} \tau(e)(1)$$

All events execute serially, with no erase, wear, overlap, cache, or communication model. Durations are chosen assumptions rather than calibrated latencies. The correctness target is one durable result for every logical ordinal. A sample may be reacquired without external side effects. Completed reacquisitions equal completed acquisition events minus 192; interrupted acquisitions are excluded from this count, although their partial durations remain in powered time. The per-event ledger preserves both completed and interrupted work, keeping replay counts distinct from completion time.

3 Complete comparisons and failure controls

Table 1 compares all twelve constructed schedules [S3]. Cut times are absolute values on a cumulative powered clock that pauses during outages. Each policy receives the same times, but different event durations can make a cut interrupt different operations. T0 has no cut. T1 uses 620 and 1730 ms; T2 uses 1110 and 2110 ms. T3 uses 390, 820, 1250, 1680, 2110, and 2540 ms; T4 uses 520, 760, 1000, 1240, 1480, and 1720 ms. T5 cuts every 300 ms from 300 through 3000 ms.

T6–T11 form the complete regular two-cut set: cuts occur at s and $2s$, with $s = 500, 600, 700, 800, 900$, and 1000 ms, respectively. These diagnostic schedules are not samples from an outage distribution. They support comparisons within this configuration, without a population average or statistical confidence interval.

Trace	Cuts	RC (ms)	AR (ms)	BS (ms)
T0	0	2448.0	2553.6	2508.0
T1	2	2821.0	2868.0	2851.0
T2	2	2813.0	2842.4	2833.0
T3	6	3825.0	3526.8	3619.0
T4	6	3781.0	3518.0	3595.0
T5	10	5449.0	4138.0	4194.0
T6	2	3061.0	2873.6	2875.0
T7	2	2873.0	2870.8	2834.0
T8	2	3073.0	2868.0	2877.0
T9	2	2885.0	2876.0	2836.0
T10	2	2697.0	2873.2	2879.0
T11	2	2897.0	2870.4	2838.0

Table 1. Powered completion times (ms) and cut counts for all twelve constructed schedules. Every policy–schedule case finishes with 192 durable logical records. Off time adds 40 ms per cut; repeated-acquisition counts and complete event histories remain available in [S3]. Values are deterministic calculations, not averages of physical trials.

BS does not remove the ordinary cost of finer persistence. Without interruption, RC finishes in 2448.0 ms, BS in 2508.0 ms, and AR in 2553.6 ms: BS pays 60.0 ms over RC. BS becomes fastest in T7, T9, and T11. In T9 it takes 2836.0 ms, compared with 2885.0 ms for RC and 2876.0 ms for AR,

differences of 49.0 and 40.0 ms. Yet the neighboring regular probe T10 reverses the result: RC needs 2697.0 ms and BS 2879.0 ms. An identical cut count is insufficient to predict which policy wins.

AR also remains competitive when replay is frequent. In T5, RC takes 5449.0 ms with 140 completed reacquisitions, BS takes 4194.0 ms with eleven, and AR takes 4138.0 ms with none. Blocks substantially reduce replay relative to rows in this case, but the simpler record policy still finishes 56.0 ms earlier. Conversely, T1 gives AR one completed reacquisition, BS two, and RC eight, yet RC finishes first at 2821.0 ms. Commit cost and the phases interrupted by later cuts participate in the outcome; the replay counter cannot serve as a substitute for the time ledger.

The supplied local validation reports sixteen passing named checks, including a finite sweep of 128 single-cut positions per policy. All 384 sweep cases reach 192 durable records. Other checks cover coordinate uniqueness, odd-row reversal, analytic no-cut totals, rejection of malformed seals, stopping at a prefix hole, and checkpoint-slot selection [S4]. These checks exercise a durable-prefix event simulator and separate byte-checksum routines. They do not form a byte-complete storage crash emulator or prove correctness for every possible sequence of cuts and corruptions.

The independent coordinate control injects a fixed +4 mm residual x displacement after reset at the Figure 1 state. It tests only the eight remaining samples of row 5. Anchoring gives zero incorrect assignments out of eight; omitting it shifts all eight by one column, from intended columns 7–0 to actual columns 8–1 [S4]. This is a prescribed arithmetic failure, not an observed drift rate. It exposes a failure that payload integrity checks cannot detect: correctly stored bytes may still be associated with an incorrect physical location.

4 Adoption conditions and limits

The storage comparison places the timing results in context. RC, AR, and BS require 4096, 256, and 1024 payload-buffer bytes respectively, excluding metadata, motion state, and runtime stack. Their simplified completed-job persistent allowances are 49280, 52224, and 50432 bytes: respectively $192 \times 256 + 2 \times 64$, 192×272 , and $48 \times 1048 + 2 \times 64$. Alignment, allocators, and filesystems are excluded. BS reduces buffering relative to RC but does not beat AR on that criterion; it reduces the persistent allowance relative to AR but remains above RC. These byte counts do not determine the assumed write latencies.

The block policy is an intermediate choice whose additional recovery work must be weighed against avoided acquisitions and publication cost. Its seals preserve partial rows while the inherited checkpoint protocol still recovers completed rows. In this configuration BS wins T7, T9, and T11, but RC wins without cuts and AR wins the ten-cut case; the regular probes change ordering with cut phase. BS also pays for lookup when no useful block survives. The evidence therefore motivates evaluating actual memory limits, small-write costs and interruption schedules together. It supplies no adoption threshold or general ranking from an intermediate commit size alone.

Several assumptions would need independent validation before deployment. The combined BS event supplies payload-before-seal ordering by construction; a physical storage implementation would need to enforce it. The checkpoint example tests generation and checksum selection but supplies neither a device driver nor a byte-exact 64-byte slot serializer. The fixed anchor duration does not model distance-dependent travel or positioning errors. The prescribed coordinate control establishes the consequence of one displacement, not the reliability of a real homing or anchoring mechanism.

The evidence covers one geometry, one record size, and one set of costs, without a latency or geometry sweep. It also assumes repeatable acquisition: a moving target, destructive probe, or external side effect would change the correctness problem because repeating a sample might no longer be acceptable. Neither the finite sweep nor the twelve profiles represents real outage behavior. Hardware timing, endurance, and energy remain unmeasured, and novelty against published systems remains open. These gaps limit generalization without obscuring the implemented tradeoff demonstrated by the accounting.

5 Conclusion

Four-record seals extend an inherited row checkpoint with recovery of a contiguous partial row. In this synthetic raster instance, they offer a smaller buffer than row commits and lower configured per-sample commit overhead than record appends, but no universal completion-time advantage. Complete trace comparisons retain the regimes where each alternative wins. The displacement control separately shows why the recovered logical boundary must be paired with position establishment. The deliverable is an inspectable model-based design analysis, with physical validity and external originality still to be established.

Internal sources

[S1] `specification.md` and `data/parameters.json`. Constructed geometry, state semantics, timing assumptions, and scope. Synthetic DEMO sources created for this package.

[S2] `implementation_log.md` and `model.py`. Local construction record and executable model. No external project history is claimed.

[S3] `data/traces.json`, `data/results.csv`, and `data/events.csv`. Deterministic trace definitions and generated accounting results.

[S4] `tests/test_results.json`, `tests/single_cut_sweep.csv`, and `data/anchor_control.csv`. Local finite checks and prescribed coordinate fault injection. These are not experimental receipts from equipment.